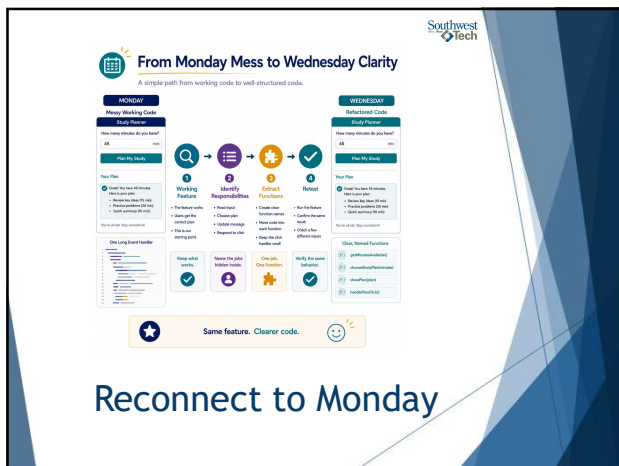




1



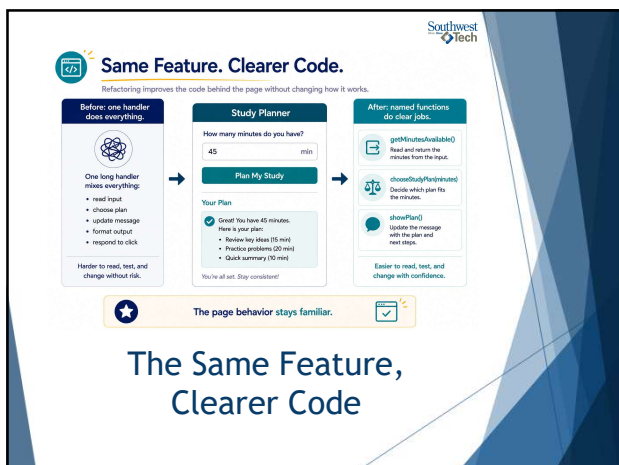
2



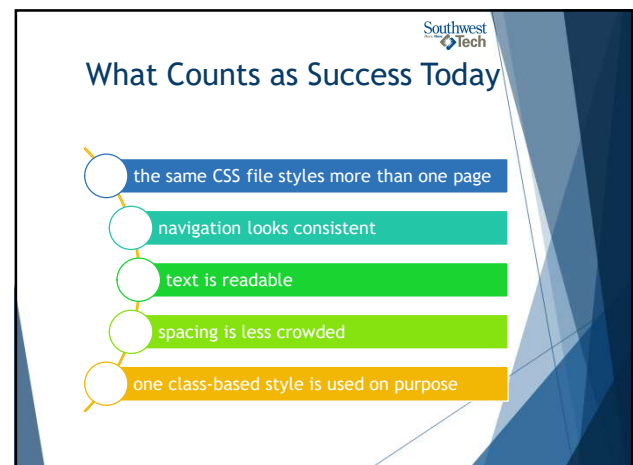
3



4



5



6

Refactor In Small Steps

Do not refactor everything at once.

Use this rhythm:

1. choose one responsibility
2. move that code into a function
3. reconnect the call
4. retest
5. then choose the next responsibility

7

Small-Step Refactoring Rhythm

Make steady progress with small, testable steps.



Do not refactor everything at once.

Small-Step Refactor

8

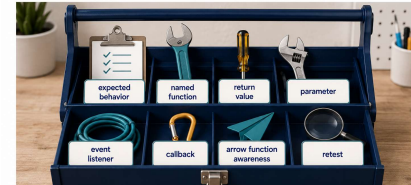
What We Will Use Today

- ▶ expected behavior
- ▶ named function
- ▶ return value
- ▶ parameter
- ▶ event listener
- ▶ callback
- ▶ arrow function awareness
- ▶ retest

9

Today's Refactoring Toolbox

Tools for turning messy working code into clear, well-structured code.



Use the right tool. One step at a time.

Refactoring Toolbox

10

Before You Change Structure, Record Behavior

Before refactoring, record expected behavior.

Example test values:

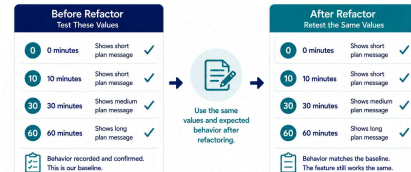
- ▶ 0 minutes
- ▶ 10 minutes
- ▶ 30 minutes
- ▶ 60 minutes

After refactoring, run the same checks.

11

Record Behavior Before Refactoring

Capture what the feature does so you can confirm it stays the same.



Retest the same values.

Record Behavior First

12

Southwest Tech

Extract One Responsibility

Look for one job:

- ▶ read input
- ▶ validate input
- ▶ choose a plan
- ▶ update the page
- ▶ handle the click

Each function should have a name that says its job.

13

Southwest Tech

Extract One Responsibility from a Long Handler

Pull out a single job into a named function.

Before: one handler does everything

```
onPlanClick()
  Read minutes from input
  Validate the minutes
  Choose a plan
  Decide which plan fits the minutes
  Update the message with the plan
  Respond to the click
```

After: one job is extracted

```
onPlanClick()
  Read minutes from input
  Validate the minutes
  Call the new function
  chooseStudyPlan(minutes)
  Update the message with the plan
  Respond to the click
```

New Function

```
chooseStudyPlan(minutes)
  Decide which plan
  fits the minutes
```

Extract one job at a time.

Extract Responsibility

14

Southwest Tech

Return Values Carry Results Back

A return value sends a result back.

Example:

```
'chooseStudyPlan(minutes)'
```

returns the message to show.

Then another function can use that result.
Returning a value keeps jobs separate.

15

Southwest Tech

Return Value Carries the Result Back

One function decides. Another function shows. The page updates.

- 1 chooseStudyPlan (minutes)**
Looks at the minutes and chooses the best study plan.
- 2 returns message**
Function completes and sends the result back.
The chosen plan message is returned to the caller.
- 3 showPlan (message)**
Returns message carries the result.
Receives the message and updates the output area.
- 4 page updates**
The user sees the study plan on the page.

Returning a value keeps jobs separate.

Return Value

16

Southwest Tech

Demo

"Refactor Into Named Functions"

17

Southwest Tech

What to Watch For

- ▶ record expected behavior
- ▶ extract `getMinutesAvailable()`
- ▶ extract `chooseStudyPlan(minutes)`
- ▶ extract `showPlan()`
- ▶ reconnect the click listener
- ▶ retest the same values

18

Refactor Into Named Functions
A steady path from working code to clearer structure.

1 record expected behavior
Capture what the page does today. These are our test values.

2 extract getMinutes Available()
Move input reading and validation into one function.

3 extract chooseStudyPlan (minutes)
Move the decision about which plan to use into its own function.

4 extract showPlan()
Move the output option into its own function.

5 reconnect click listener
Wire the button click to call the first function.

6 retest
Run the same test values. Confirm the page still behaves the same.

Structure changes. Behavior stays verified.

Demo: Refactor

19

Named Function Callback

Before:

```
button.addEventListener("click", function () { ... });
```

After:

```
button.addEventListener("click", showPlan);
```

The named function makes the event's purpose visible.

20

Anonymous Callback vs. Named Callback
Same event. Clearer intent.

Before: Anonymous Callback

```
addEventListener("click", function () {  
  // your code here ...  
});
```

What does this do?
The code runs, but the purpose is not obvious.

After: Named Callback

```
addEventListener("click", showPlan);
```

What does this do?
The name shows the purpose: show the study plan.

The named function makes the event's purpose visible.

Named Function Callback

21

Arrow Functions Are Alternate Syntax

Arrow functions are another way to write functions.

Traditional:

```
function showPlan() { ... }
```

Arrow style:

```
const showPlan = () => { ... };
```

For this course:

- recognize arrow functions.
- Use clear names first.

22

Two Ways You May See a Function
Same job. Different syntax styles.

Traditional Function Syntax
A style you have seen.

```
function showPlan() {  
  // decide the plan  
  ...  
}
```

Defines a function with a name. Use it. Call it. Read it.

Arrow Function Syntax
An alternative style you may see.

```
const showPlan = () => {  
  // decide the plan  
  ...  
};
```

Also defines a function. Often used for short, clear jobs.

Both styles can do the same job.

Recognize arrow functions. Use clear names first.

Arrow Function Awareness

23

Why Arrow Functions Became Common

Advantages:

- shorter syntax for small functions
- common in modern examples
- useful for callbacks and array methods
- avoids some 'this' confusion in advanced code

Disadvantages:

- can be harder to read at first
- not ideal for every named function
- easy to overuse for clever one-liners

Recommendation:

- start using arrow functions for small callbacks,
- but keep names and readability first.

24

Southwest Tech

Messy Code vs. Cleaner Code

Same feature. Very different clarity.

Messy: purpose hidden.
One long block tries to do everything.

```

1 function chooseStudyPlan(minutes) {
2   const value = parameter(input.value);
3   return value > 0 ? value : 0;
4 }
5
6 function showPlan() {
7   // ...
8   // ...
9   // ...
10  // ...
11 }

```

Cleaner: purpose revealed.
Small functions, clear jobs.

```

getMinutesAvailable()
Read and validate the minutes.

chooseStudyPlan(minutes)
Decide which plan fits the minutes.

showPlan()
Update the page with the plan.

```

responsibilities visible
Each function has one clear job.

testing points clearer
Each function can be tested on its own.

page update easier to find
The output logic lives in one obvious place.

Cleaner code is easier to explain.

Why Arrow Functions Became Common

25

Southwest Tech

Clean Versus Messy Comparison

Messy code hides:

- ▶ purpose
- ▶ responsibilities
- ▶ testing points

Cleaner code reveals:

- ▶ what each function does
- ▶ where values come from
- ▶ where the page updates
- ▶ what to retest

26

Southwest Tech

Messy Code vs. Cleaner Code

Same feature. Very different clarity.

Messy: purpose hidden.
One long block tries to do everything.

```

1 function chooseStudyPlan(minutes) {
2   const value = parameter(input.value);
3   return value > 0 ? value : 0;
4 }
5
6 function showPlan() {
7   // ...
8   // ...
9   // ...
10  // ...
11 }

```

Cleaner: purpose revealed.
Small functions, clear jobs.

```

getMinutesAvailable()
Read and validate the minutes.

chooseStudyPlan(minutes)
Decide which plan fits the minutes.

showPlan()
Update the page with the plan.

```

responsibilities visible
Each function has one clear job.

testing points clearer
Each function can be tested on its own.

page update easier to find
The output logic lives in one obvious place.

Cleaner code is easier to explain.

Clean vs. Messy Comparison

27

Southwest Tech

AI Can Explain Choices, Not Decide For You

Useful for:

- ▶ comparing two function names
- ▶ explaining a return value
- ▶ explaining why a callback works
- ▶ identifying what a function is responsible for

You still:

- ▶ Choose
- ▶ Test
- ▶ explain.

28

Southwest Tech

AI Explains Structure Choices—Not Replaces You

You write the code. AI helps you understand and decide.

Your Code (Student Work)
A small excerpt from your project.

```

1 function getMinutesAvailable() {
2   const value = parameter(input.value);
3   return value > 0 ? value : 0;
4 }
5
6 function chooseStudyPlan(minutes) {
7   if (minutes >= 60) return "Sleep Study";
8   if (minutes >= 30) return "Focused Session";
9   return "Quick Review";
10 }
11
12 function showPlan(message) {
13   output.textContent = message;
14 }
15

```

AI Explanation (Not Replacement)
AI helps clarify your structure choices.

- Explains each function's responsibility
AI describes the job of each function in plain language so you can see the separation of responsibilities more clearly.
- Compares two function names
AI compares name options (eg., getMinutesAvailable vs. readMinutes, or showPlan vs. displayPlan) and explains the trade-offs.
- Explains a return value
AI explains what a function returns, how the value is used by the caller, and why that keeps the code clear and maintainable.

Boundary: You still choose, test, and explain.
AI supports your thinking. Your choices drive the code.

AI Explains Choices

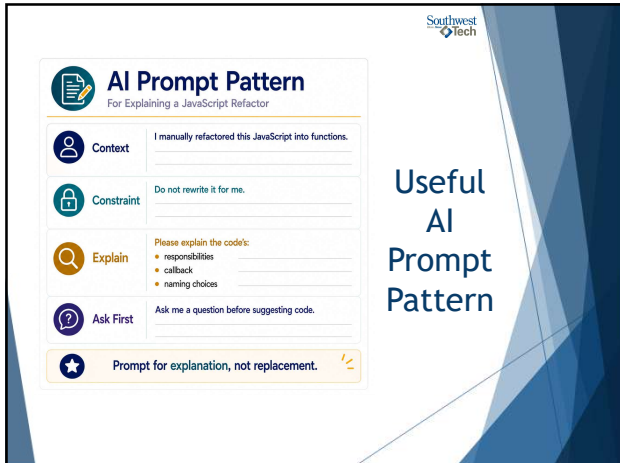
29

Southwest Tech

Useful AI Prompt Pattern

"I manually refactored this JavaScript into functions. Do not rewrite it for me. Explain what each function is responsible for, where the callback happens, and one name that could be clearer. Ask me a question before suggesting code."

30



AI Prompt Pattern
For Explaining a JavaScript Refactor

Context I manually refactored this JavaScript into functions.

Constraint Do not rewrite it for me.

Explain Please explain the code's:
 • responsibilities
 • callback
 • naming choices

Ask First Ask me a question before suggesting code.

★ Prompt for explanation, not replacement.

Useful AI Prompt Pattern

31



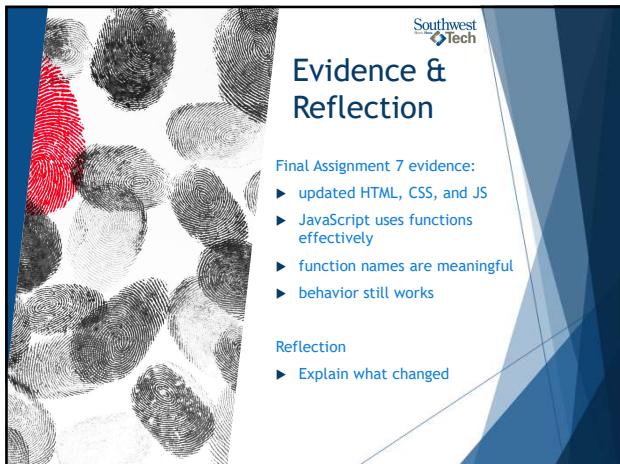
Lab Refinement

Refine structure

Your goal:

- ▶ improve function names
- ▶ organize logic clearly
- ▶ reduce repetition where possible
- ▶ keep the feature working
- ▶ add one clarity improvement

32



Evidence & Reflection

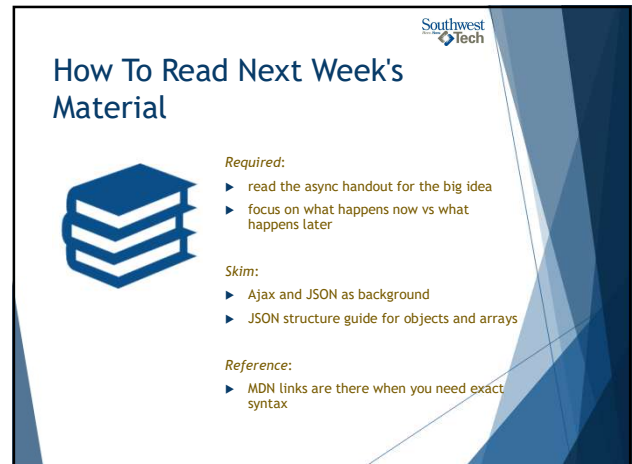
Final Assignment 7 evidence:

- ▶ updated HTML, CSS, and JS
- ▶ JavaScript uses functions effectively
- ▶ function names are meaningful
- ▶ behavior still works

Reflection

- ▶ Explain what changed

33



How To Read Next Week's Material

Required:

- ▶ read the async handout for the big idea
- ▶ focus on what happens now vs what happens later

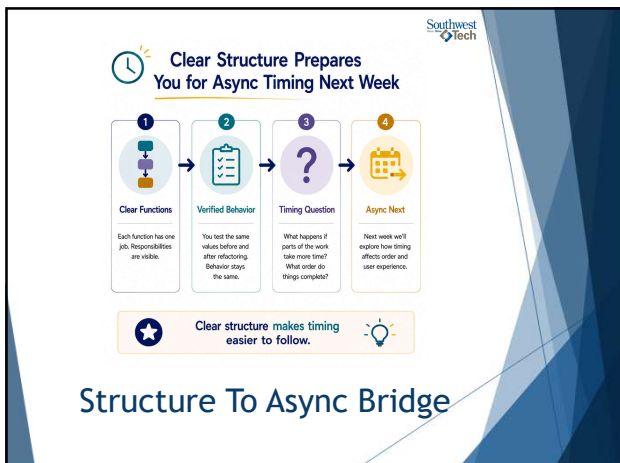
Skim:

- ▶ Ajax and JSON as background
- ▶ JSON structure guide for objects and arrays

Reference:

- ▶ MDN links are there when you need exact syntax

34



Clear Structure Prepares You for Async Timing Next Week

- 1 Clear Functions**
Each function has one job. Responsibilities are visible.
- 2 Verified Behavior**
You test the same values before and after refactoring. Behavior stays the same.
- 3 Timing Question**
What happens if parts of the work take more time? What order do things complete?
- 4 Async Next**
Next week we'll explore how timing affects order and user experience.

★ Clear structure makes timing easier to follow.

Structure To Async Bridge

35



Closing Success Check

This week:

- ▶ working code became clearer
- ▶ responsibilities became functions
- ▶ callbacks became easier to read
- ▶ behavior stayed verified

Next:

- ▶ systems respond over time.

36



37